Minijuego: “Piedra, Papel o Tijera” con Persistencia de datos (struct + fstream)

Alexander Luca Veit
E.E.S.T N°1 de Vicente López
Curso: Laboratorio de Programación 5°3
Profesor: Mansilla Muñoz York E.
Fecha: 22/10/2026

Introducción

Este trabajo tiene como objetivo presentar la estructura general de un informe académico siguiendo las normas del estilo APA (7ª edición). Aquí se desarrollarán los contenidos trabajados durante la actividad, incluyendo descripciones, análisis, y conclusiones sobre el uso de entornos de desarrollo y conceptos básicos de programación.

Desarrollo

Aquí se debe incluir el desarrollo de los temas: entornos de desarrollo (Visual Studio Code, Visual Studio Community, Sublime Text), conceptos de variables, estructuras de control, operadores, tipos de datos, compilación, y ejemplos de código. Cada sección puede llevar subtítulos si el estudiante lo desea.

Respuestas al cuestionario técnico de la actividad:

1) La RAM es volátil: al cerrar la consola, la memoria se libera y todas las variables (incluido contadorID y cualquier dato en memoria) se pierden. En cambio, guardarEnDisco() escribe cada partida en historial_partidas.txt usando ofstream, que persiste en el disco duro (no volátil). Al abrir el ejecutable, la opción 2 llama a leerHistorialDisco(), que abre ese mismo archivo con ifstream y lo lee línea por línea — por eso el historial sigue intacto.
2) El modo por defecto de ofstream al abrir un archivo existente es ios::trunc, que trunca el archivo a cero (lo borra) en el momento de abrirlo. Si escribieras ofstream archivo("src/historial_partidas.txt"); sin incluir ios::app, cada vez que se llamara a guardarEnDisco() el archivo se vaciaria y solo quedaria la ultima partida. Con ios::app, cada escritura se agrega al final sin destruir el contenido previo, permitiendo que el historial crezca partida a partida.
3) Cuando el usuario elige la opción del menú con cin >> opcion, el carácter de nueva línea (\n) que presiona al pulsar Enter queda residual en el búfer de entrada. Si getline(cin, p.jugador) se ejecuta inmediatamente, detecta ese \n como "fin de línea" y devuelve una cadena vacía sin esperar al usuario. cin.ignore() descarta ese carácter residual, dejando el búfer limpio para que getline realmente espere la entrada del nombre.
4) La estructura Partida agrupa los 5 campos de una partida en una sola unidad, lo que simplifica el código: en vez de pasar 5 variables sueltas a cada función, se pasa un único argumento (const Partida& p), lo que hace las firmas más cortas, mejora la legibilidad al usar nombres como p.eleccionJugador, y facilita el mantenimiento porque si se agrega un campo nuevo solo se modifica la struct sin tocar la firma de cada función.

5) Si implementamos este módulo de almacenamiento, nosotros guardaremos un registro de las partidas jugadas (por el usuario) además de guardar la cantidad de veces que haya muerto el personaje en el juego por nivel (básicamente que se guarden las estadísticas del usuario por nivel).

Conclusión

En esta sección se resumen las principales ideas trabajadas y se expresan opiniones personales sobre lo aprendido en la actividad. Es importante incluir reflexiones sobre el uso de herramientas y el inicio en la programación.

En esta séptima actividad, se hizo un programa que ejecutaba el clásico juego de piedra papel o tijera contra una IA. Además de que se implementó una función en el programa para poder crear un historial de partidas y almacenarlas dentro de un archivo .txt. También se tuvo en cuenta el dual-remote (tener un repositorio backup en gitlab).

Referencias

Alexander, L. Veit (2026) Programador Junior [Código de Software]

Github: https://github.com/Xen-alt10/repasogeneral/tree/main

Páginas. code.visualstudio.com (2026). Using GCC with MinGW.
https://code.visualstudio.com/docs/cpp/config-mingw